

Linux Shell Commands and System Calls

1. Overview

Linux shell commands are text-based instructions used to interact with a Linux operating system. The shell is a command-line program that reads commands, expands them, starts programs, connects input/output, and reports results.

System calls are the low-level interface through which user programs ask the operating system kernel to perform privileged work such as reading files, creating processes, opening network connections, and changing permissions.

Definition

Term	Meaning
Shell	A command interpreter such as <code>bash</code> , <code>zsh</code> , or <code>sh</code> .
Command	A program or shell built-in invoked from the terminal.
Terminal	The interface where you type commands and see output.
Kernel	Core OS component managing hardware, processes, memory, files, and networking.
System call	Controlled entry point from user mode into kernel mode.

Why It Matters

For SDE placements and backend roles, Linux shell knowledge is useful for:

- Navigating servers and logs.
- Debugging production issues.
- Running builds, tests, deployments, and scripts.
- Understanding how programs interact with the OS.
- Explaining processes, files, permissions, signals, and networking.

Where It Is Used in Real Systems

- Backend servers: checking logs, process status, ports, and configuration.
- Databases: inspecting files, permissions, backups, and service processes.
- Cloud machines: SSH access, file transfer, package downloads, and deployment.
- CI/CD pipelines: shell scripts for build, test, and release steps.
- Containers: debugging inside Docker/Linux containers.
- Operating systems: system calls power file, process, memory, and network operations.

Why Interviewers Ask About It

Interviewers ask Linux and shell questions because they reveal whether a candidate can work on real engineering systems, not only write code in an IDE. They often test:

- File navigation and search.
- Permissions and ownership.
- Process management.
- Debugging with logs.
- Networking basics.
- Difference between shell commands and OS internals.
- Understanding of common system calls like `fork`, `exec`, `open`, `read`, `write`, and `kill`.

2. Core Idea

The core idea is: the shell is a user-friendly command layer over programs and operating system services. Commands like `ls`, `cat`, `grep`, and `ps` are usually normal programs. Some commands like `cd` are shell built-ins because they must modify the shell's own state.

Under the hood, these commands rely on system calls.

Intuition

Think of Linux like an office:

Office Analogy	Linux Concept
Reception desk	Shell
Employee requests	Commands
Manager with authority	Kernel
Official request forms	System calls
Filing cabinets	File system
Running workers	Processes

You do not directly touch the filing cabinet or power system. You ask the authorized manager through official request forms. Similarly, a program cannot directly access disk, memory, or network hardware. It asks the kernel using system calls.

Small Example

Command:

```
cat notes.txt
```

What happens conceptually:

1. The shell parses the command.
2. The shell finds the `cat` executable.
3. The shell creates a child process using a system call such as `fork`.
4. The child process replaces itself with the `cat` program using `exec`.

5. `cat` opens `notes.txt` using `open`.
6. `cat` reads file data using `read`.
7. `cat` writes data to terminal using `write`.
8. The process exits using `exit`.
9. The shell waits and shows the prompt again.

Step-by-Step Mental Model

```
graph TD; A[User types command] --> B[Shell parses command]; B --> C[Shell starts program]; C --> D[Program requests kernel services using system calls]; D --> E[Kernel accesses files/processes/network safely]; E --> F[Output returns to terminal];
```

3. Important Subtopics

3.1 `pwd` - Print Working Directory

What it means:

`pwd` prints the current directory path.

```
pwd
```

Example output:

```
/home/student/projects
```

Why it matters:

Many commands depend on the current directory. If you run a script from the wrong directory, relative paths may break.

Example:

```
cd /var/log
pwd
```

Common interview angle:

Difference between absolute and relative paths.

Path Type	Example	Meaning
Absolute	/home/user/file.txt	Starts from root /.
Relative	docs/file.txt	Starts from current directory.

3.2 cd - Change Directory

What it means:

cd changes the shell's current working directory.

```
cd /etc
cd ..
cd ~
cd -
```

Why it matters:

Navigation is the base of shell work.

Example:

```
cd /var/log
less syslog
```

Common interview angle:

Why is cd a shell built-in?

Answer: cd must change the current directory of the running shell process itself. If it were only an external child process, it would change the child's directory and then exit, leaving the parent shell unchanged.

3.3 ls - List Directory Contents

What it means:

ls lists files and directories.

```
ls
ls -l
ls -a
```

```
ls -lh
ls -ltr
```

Option	Meaning
--------	---------

-l	Long listing with permissions, owner, size, time.
-a	Show hidden files.
-h	Human-readable sizes.
-t	Sort by modification time.
-r	Reverse order.

Example:

```
ls -lah /var/log
```

Common interview angle:

Explain file permission output.

```
-rw-r--r-- 1 alice dev 1200 Jul 9 app.log
```

Part	Meaning
------	---------

-	Regular file.
rw-	Owner can read/write.
r--	Group can read.
r--	Others can read.

3.4 cat - Display or Concatenate Files

What it means:

cat prints file contents to standard output or combines files.

```
cat file.txt
cat a.txt b.txt
```

Why it matters:

Useful for small files, config inspection, and pipeline input.

Example:

```
cat /etc/hosts
```

Common interview angle:

When should you not use `cat`?

For large files, use `less`, `head`, `tail`, or streaming tools. `cat huge.log` can flood the terminal.

3.5 `less` - View Large Files Page by Page

What it means:

`less` opens a file in a pager.

```
less app.log
```

Useful keys:

Key	Action
Space	Next page
b	Previous page
/text	Search forward
n	Next search result
q	Quit

Why it matters:

Production logs can be huge. `less` allows safe inspection without loading everything onto the screen.

Example:

```
less /var/log/nginx/access.log
```

Common interview angle:

Difference between `cat` and `less`.

3.6 `grep` - Search Text Patterns

What it means:

`grep` searches text for matching lines.

```
grep "ERROR" app.log  
grep -i "error" app.log
```

```
grep -r "TODO" .  
grep -n "main" file.c
```

Option	Meaning
-i	Case-insensitive
-r	Recursive search
-n	Show line numbers
-v	Invert match
-E	Extended regular expressions

Why it matters:

It is one of the fastest ways to inspect logs, source code, configs, and command output.

Example:

```
grep -i "failed" /var/log/auth.log
```

Common interview angle:

How would you find all failed login attempts in logs?

3.7 find - Search Files by Name, Type, Size, Time

What it means:

find searches the file system using conditions.

```
find . -name "*.log"  
find /var/log -type f -mtime -1  
find . -type f -size +100M
```

Expression	Meaning
-name	Match file name
-type f	Regular files
-type d	Directories
-mtime -1	Modified in last 1 day
-size +100M	Larger than 100 MB

Why it matters:

Useful when files are spread across directories.

Example:

```
find . -type f -name "*.java"
```

Common interview angle:

Difference between `grep` and `find`.

Tool	Searches
<code>find</code>	File names, metadata, paths
<code>grep</code>	File contents

3.8 `chmod` - Change File Permissions

What it means:

`chmod` changes read, write, and execute permissions.

```
chmod 755 script.sh
chmod +x script.sh
chmod u+w file.txt
chmod go-r secret.txt
```

Permission values:

Permission	Symbol	Number
Read	<code>r</code>	4
Write	<code>w</code>	2
Execute	<code>x</code>	1

Numeric examples:

Mode	Meaning
<code>777</code>	Everyone can read, write, execute. Usually unsafe.
<code>755</code>	Owner full access, others read/execute. Common for scripts/directories.
<code>644</code>	Owner read/write, others read. Common for files.
<code>600</code>	Only owner read/write. Common for private keys.

Why it matters:

Permissions protect files and control executable scripts.

Example:

```
chmod 600 ~/.ssh/id_rsa
```

Common interview angle:

Why must SSH private keys often be 600?

Because private keys should not be readable by group or others. SSH may reject insecure key permissions.

3.9 ps - Show Processes

What it means:

ps shows currently running processes.

```
ps
ps aux
ps -ef
```

Why it matters:

Used to debug running services, stuck programs, CPU usage, and process IDs.

Example:

```
ps aux | grep nginx
```

Common columns:

Column	Meaning
PID	Process ID
PPID	Parent process ID
USER	Process owner
CPU	CPU usage
MEM	Memory usage
CMD	Command

Common interview angle:

What is a PID? What is a parent process?

3.10 kill - Send Signals to Processes

What it means:

kill sends a signal to a process. It does not always mean "forcefully terminate."

```
kill 1234
kill -9 1234
kill -15 1234
```

Common signals:

Signal	Number	Meaning
SIGTERM	15	Graceful termination request
SIGKILL	9	Force kill, cannot be caught
SIGHUP	1	Hangup/reload in many daemons
SIGINT	2	Interrupt, like Ctrl+C

Why it matters:

Backend engineers often need to stop or restart stuck processes.

Example:

```
kill -15 4321
```

If it does not stop:

```
kill -9 4321
```

Common interview angle:

Difference between `kill -15` and `kill -9`.

3.11 curl - Transfer Data Using URLs

What it means:

`curl` sends requests to URLs and prints responses.

```
curl https://example.com
curl -I https://example.com
curl -X POST -H "Content-Type: application/json" -d '{"name":"Asha"}'
http://localhost:3000/users
```

Option	Meaning
-I	Fetch headers only
-X	Set HTTP method

Option	Meaning
-H	Add header
-d	Send request body
-o	Save output to file

Why it matters:

Used to test APIs, debug HTTP responses, check headers, and call services.

Example:

```
curl -I https://api.github.com
```

Common interview angle:

How do you test whether a backend endpoint is working?

3.12 `wget` - Download Files from the Web

What it means:

`wget` downloads files from URLs.

```
wget https://example.com/file.zip  
wget -O output.zip https://example.com/file.zip
```

Why it matters:

Useful for downloading packages, datasets, archives, or static files on servers.

Example:

```
wget https://example.com/app.tar.gz
```

Common interview angle:

Difference between `curl` and `wget`.

Tool	Best For
<code>curl</code>	API testing, custom HTTP requests, protocol debugging
<code>wget</code>	Simple recursive and resilient file downloads

3.13 `ssh` - Secure Remote Login

What it means:

`ssh` securely connects to a remote machine.

```
ssh user@server.example.com
ssh -i key.pem ubuntu@1.2.3.4
```

Why it matters:

Cloud servers are commonly accessed using SSH.

Example:

```
ssh ubuntu@192.168.1.10
```

Common interview angle:

What happens when you SSH into a server?

Key points:

- Client contacts server on port 22 by default.
- Server proves its identity using a host key.
- User authenticates using password or public/private key.
- Traffic is encrypted.

3.14 scp - Secure Copy

What it means:

scp copies files between local and remote machines over SSH.

```
scp file.txt user@server:/home/user/
scp user@server:/var/log/app.log .
scp -r folder user@server:/tmp/
```

Why it matters:

Useful for transferring logs, configs, builds, and backups.

Example:

```
scp app.tar.gz ubuntu@1.2.3.4:/home/ubuntu/
```

Common interview angle:

Difference between `ssh` and `scp`.

Command	Purpose
ssh	Remote shell login
scp	File transfer over SSH

3.15 tar - Archive and Extract Files

What it means:

tar groups multiple files into one archive. With compression options, it can also create compressed archives.

```
tar -cvf archive.tar folder/  
tar -xvf archive.tar  
tar -czvf archive.tar.gz folder/  
tar -xzvf archive.tar.gz
```

Option	Meaning
-c	Create archive
-x	Extract archive
-v	Verbose
-f	File name follows
-z	Use gzip compression
-t	List archive contents

Why it matters:

Used for backups, deployment bundles, log packaging, and source releases.

Example:

```
tar -czvf logs.tar.gz /var/log/myapp
```

Common interview angle:

Difference between archiving and compression.

Concept	Meaning
Archive	Combines many files into one file.
Compression	Reduces file size.
tar	Archives files.
gzip	Compresses data.
.tar.gz	Tar archive compressed with gzip.

3.16 Shell Redirection and Pipes

What it means:

Redirection changes where input/output goes. Pipes connect output of one command to input of another.

```
command > file.txt
command >> file.txt
command < input.txt
command 2> error.log
command1 | command2
```

Syntax	Meaning
>	Write stdout to file, overwrite
>>	Append stdout to file
2>	Write stderr to file
&>	Write stdout and stderr

Example:

```
ps aux | grep java
```

Common interview angle:

What is the difference between stdout and stderr?

3.17 Environment Variables

What it means:

Environment variables are key-value settings available to processes.

```
echo $PATH
export PORT=8080
```

Why it matters:

Used for configuration, secrets, executable search paths, and deployment settings.

Example:

```
export NODE_ENV=production
```

Common interview angle:

What is `PATH`?

`PATH` is a colon-separated list of directories the shell searches when you type a command.

3.18 Important System Calls

System calls are not shell commands, but shell commands depend on them.

System Call	Purpose	Example Command Using It
<code>open</code>	Open a file	<code>cat file.txt</code>
<code>read</code>	Read data from file descriptor	<code>cat</code> , <code>grep</code>
<code>write</code>	Write data to file descriptor	<code>echo</code> , <code>cat</code>
<code>close</code>	Close file descriptor	Most file commands
<code>stat</code>	Get file metadata	<code>ls -l</code>
<code>chdir</code>	Change current directory	<code>cd</code>
<code>chmod</code>	Change file mode	<code>chmod</code>
<code>fork</code>	Create child process	Shell running commands
<code>exec</code>	Replace process image	Shell starting <code>ls</code>
<code>wait</code>	Wait for child process	Shell after command
<code>exit</code>	Terminate process	Any program ending
<code>kill</code>	Send signal	<code>kill</code>
<code>pipe</code>	Create pipe	<code>`cmd1</code>
<code>dup2</code>	Redirect file descriptors	<code>cmd > out.txt</code>
<code>socket</code>	Create network endpoint	<code>curl</code> , <code>ssh</code>
<code>connect</code>	Connect to remote host	<code>curl</code> , <code>ssh</code>
<code>accept</code>	Accept incoming connection	Web server
<code>mmap</code>	Map file/memory into address space	Databases, loaders

Common interview angle:

Explain what happens when a shell runs `ls`.

Expected answer: shell parses command, forks child, child execs `ls`, `ls` uses system calls like `opendir`, `readdir`, `stat`, and `write`, parent shell waits.

4. Real-World Example

Debugging a Backend Server Issue

Suppose a Java backend API is slow or failing on a Linux server.

Step 1: SSH into the server.

```
ssh ubuntu@api-server
```

Step 2: Check current directory.

```
pwd
```

Step 3: Move to log directory.

```
cd /var/log/myapp
```

Step 4: List latest logs.

```
ls -ltr
```

Step 5: View the newest log safely.

```
less app.log
```

Step 6: Search errors.

```
grep -i "exception" app.log  
grep -i "timeout" app.log
```

Step 7: Find large or recent files.

```
find . -type f -size +100M  
find . -type f -mtime -1
```

Step 8: Check running process.

```
ps aux | grep java
```

Step 9: Test API locally.

```
curl -I http://localhost:8080/health
```

Step 10: Gracefully stop the process if needed.

```
kill -15 <PID>
```

Step 11: Package logs for investigation.

```
tar -czvf debug-logs.tar.gz *.log
```

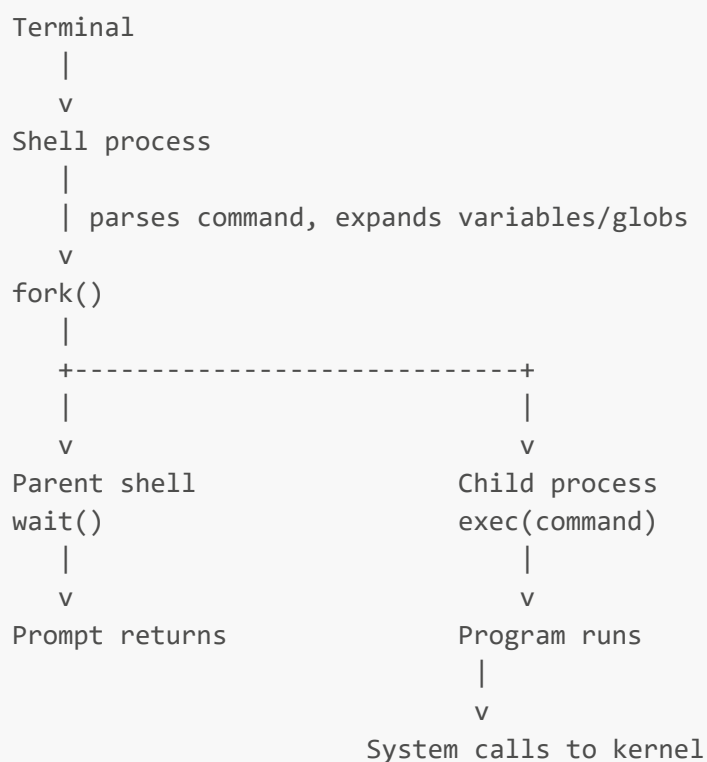
Step 12: Copy logs to local machine.

```
scp ubuntu@api-server:/var/log/myapp/debug-logs.tar.gz .
```

This flow is common in production debugging, DevOps tasks, support engineering, and backend development.

5. Diagrams / Mental Models

Shell Command Execution



User Mode vs Kernel Mode

```

User Mode
-----
Shell, cat, grep, curl, browser, database
    |
    | system call
    v
Kernel Mode
-----
File system, process scheduler, memory manager, network stack, device drivers

```

File Descriptor Mental Model

Every process starts with three standard file descriptors.

FD	Name	Default Destination
0	stdin	Keyboard/input
1	stdout	Terminal output
2	stderr	Terminal error output

Redirection changes these mappings.

```
cat file.txt > out.txt
```

Before:

```
stdout -> terminal
```

After:

```
stdout -> out.txt
```

Pipeline Mental Model

```
ps aux | grep java
```

```

ps aux stdout ----pipe----> grep stdin
grep stdout -----> terminal

```

Permission Model

```

-  rwx  rwx  rwx
|  |    |    |
|  |    |    +-- others
|  |    +----- group
|  +----- owner
+----- file type

```

6. Common Interview Questions

1. What is a shell?

Answer:

A shell is a command interpreter that lets users interact with the operating system by typing commands. It parses input, expands variables and wildcards, starts programs, manages pipes/redirection, and returns command output.

Interviewer expects:

- Command interpreter.
- Interface between user and OS utilities.
- Can run scripts.
- Examples: `bash`, `zsh`, `sh`.

Common mistakes:

- Saying shell and terminal are exactly the same.
- Saying shell is the kernel.

2. What is the difference between terminal, shell, and kernel?

Answer:

The terminal is the input/output interface. The shell is the program that interprets commands. The kernel is the core OS component that manages hardware and resources.

Interviewer expects:

- Terminal displays interaction.
- Shell parses commands.
- Kernel executes privileged operations through system calls.

Common mistakes:

- Calling every command-line window a shell.
- Ignoring the kernel's role.

3. What happens when you type `ls` and press Enter?

Answer:

The shell parses `ls`, finds the executable using `PATH`, creates a child process using `fork`, the child runs `ls`

using `exec`, `ls` reads directory entries and metadata using system calls, writes output to stdout, exits, and the parent shell waits before showing the prompt.

Interviewer expects:

- Parse.
- PATH lookup.
- `fork`.
- `exec`.
- System calls.
- `wait`.

Common mistakes:

- Saying the shell itself lists the files in every case.
- Missing process creation.

4. Why is `cd` usually a shell built-in?

Answer:

`cd` changes the current working directory of the shell process. If `cd` ran only as a separate external program, it would change the child process's directory and then exit, leaving the parent shell unchanged.

Interviewer expects:

- Parent process state.
- Child cannot directly change parent's current directory.

Common mistakes:

- Saying it is built-in only for speed.

5. What is the difference between `cat` and `less`?

Answer:

`cat` prints the whole file to stdout, useful for small files and pipelines. `less` opens a file page by page, useful for large files and interactive searching.

Interviewer expects:

- `cat` streams to stdout.
- `less` is interactive pager.
- Large-file safety.

Common mistakes:

- Using `cat` for huge logs in production.

6. What is the difference between `grep` and `find`?

Answer:

`grep` searches file contents or input text. `find` searches files/directories by name, type, size, time, ownership,

and other metadata.

Interviewer expects:

- Content search vs file metadata/path search.

Common mistakes:

- Saying both only search file names.

7. Explain Linux file permissions.

Answer:

Linux permissions define read, write, and execute access for three categories: owner, group, and others. For example, `chmod 755 script.sh` means owner has read/write/execute, group has read/execute, and others have read/execute.

Interviewer expects:

- `r, w, x`.
- Owner/group/others.
- Numeric representation: 4, 2, 1.
- Directory execute means ability to enter/traverse.

Common mistakes:

- Forgetting directory permissions behave differently from file permissions.
- Saying `777` is generally safe.

8. What is the difference between `kill -15` and `kill -9`?

Answer:

`kill -15` sends `SIGTERM`, asking a process to terminate gracefully. The process can catch it and clean up. `kill -9` sends `SIGKILL`, which cannot be caught or ignored and forcefully terminates the process.

Interviewer expects:

- `SIGTERM` graceful.
- `SIGKILL` forceful.
- Try `-15` before `-9`.

Common mistakes:

- Saying `kill` always force kills.

9. What are stdin, stdout, and stderr?

Answer:

They are standard file descriptors opened for every process. `stdin` is input, `stdout` is normal output, and `stderr` is error output.

Interviewer expects:

- FD 0, 1, 2.
- Redirection examples.

Common mistakes:

- Mixing stdout and stderr.
- Not knowing `2>` redirects stderr.

10. What is a pipe?

Answer:

A pipe connects the stdout of one process to the stdin of another process.

Example:

```
ps aux | grep python
```

Here, output from `ps aux` becomes input to `grep python`.

Interviewer expects:

- Inter-process communication.
- File descriptor redirection.
- Streaming behavior.

Common mistakes:

- Thinking the first command must finish completely before the second starts.

11. What is the difference between `curl` and `wget`?

Answer:

`curl` is commonly used for API requests, HTTP debugging, custom headers, methods, and request bodies.

`wget` is commonly used for downloading files, including recursive or resumable downloads.

Interviewer expects:

- `curl` for requests.
- `wget` for downloads.

Common mistakes:

- Saying they are fully identical.

12. What is SSH?

Answer:

SSH is a secure protocol for remote login and command execution over an encrypted connection. It commonly uses port 22 and supports password or public-key authentication.

Interviewer expects:

- Secure remote access.
- Encryption.
- Public/private key authentication.
- Host key verification.

Common mistakes:

- Confusing SSH with HTTP.
- Thinking the public key must be secret.

13. What is the difference between `tar`, `gzip`, and `.tar.gz`?

Answer:

`tar` archives multiple files into one file. `gzip` compresses data. A `.tar.gz` file is a tar archive compressed with `gzip`.

Interviewer expects:

- Archive vs compression.
- Common commands for create/extract.

Common mistakes:

- Saying `tar` always compresses by itself.

14. What is a system call?

Answer:

A system call is an interface through which a user-space program requests a service from the kernel, such as file access, process creation, networking, or memory management.

Interviewer expects:

- User mode to kernel mode.
- Controlled privileged operation.
- Examples: `open`, `read`, `write`, `fork`, `exec`, `wait`, `kill`.

Common mistakes:

- Confusing library functions with system calls.

15. What is the difference between `fork` and `exec`?

Answer:

`fork` creates a new child process by duplicating the current process. `exec` replaces the current process image with a new program. Shells commonly use `fork` followed by `exec` to run external commands.

Interviewer expects:

- `fork` creates.
- `exec` replaces.
- Used together by shell.

Common mistakes:

- Saying `exec` creates a new process.

7. Deep-Dive Questions

1. How does shell redirection work internally?

Answer:

The shell opens the target file using a system call like `open`, then uses `dup2` to make `stdout` or `stderr` point to that file descriptor. After that, when the child program writes to `stdout`, the data goes to the file instead of the terminal.

Example:

```
ls > out.txt
```

Conceptually:

```
open("out.txt") -> fd 3
dup2(3, 1)      -> stdout now points to out.txt
exec("ls")
```

2. Why can a process become a zombie?

Answer:

A zombie process is a terminated child process whose exit status has not yet been collected by its parent using `wait`. The process is no longer running, but its entry remains in the process table until the parent reaps it.

Key points:

- Zombie is already dead, not consuming CPU.
- Parent must call `wait`.
- Too many zombies can exhaust process table entries.

3. How do pipes work at the OS level?

Answer:

A pipe is a kernel-managed buffer with a read end and write end. The shell creates a pipe using `pipe`, forks processes, connects one process's `stdout` to the write end and another process's `stdin` to the read end using `dup2`, then runs both commands.

Example:

```
cat app.log | grep ERROR
```

```
cat stdout -> pipe write end -> pipe buffer -> pipe read end -> grep stdin
```

4. Why should `kill -9` be avoided as the first option?

Answer:

`kill -9` prevents the target process from handling the signal. It cannot close files gracefully, flush buffers, release locks cleanly, or notify other services. `SIGTERM` gives the process a chance to shut down properly.

Use order:

1. Try normal service stop command.
2. Try `kill -15`.
3. Use `kill -9` only if necessary.

5. What is the difference between a shell built-in and an external command?

Answer:

A shell built-in is implemented inside the shell process itself. An external command is a separate executable file that the shell runs as a child process.

Examples:

Built-in	External Command
<code>cd</code>	<code>ls</code>
<code>export</code>	<code>grep</code>
<code>alias</code>	<code>curl</code>

Built-ins are needed when the command must modify shell state, such as current directory or environment variables.

8. Comparison Tables

Shell vs Terminal vs Kernel

Concept	Role	Example
Terminal	Interface for input/output	GNOME Terminal, Windows Terminal
Shell	Parses and runs commands	<code>bash</code> , <code>zsh</code>
Kernel	Manages OS resources	Linux kernel

Absolute Path vs Relative Path

Feature	Absolute Path	Relative Path
Starts from	Root /	Current directory
Example	<code>/home/user/app.log</code>	<code>logs/app.log</code>
Stable across directories	Yes	No
Common use	Scripts, config paths	Quick navigation

cat vs less

Feature	cat	less
Use case	Small files, pipelines	Large files, log viewing
Interaction	Non-interactive	Interactive
Search inside file	No built-in session search	Yes, with <code>/pattern</code>
Output	Prints all content	Page by page

grep vs find

Feature	grep	find
Searches	Text contents	File paths and metadata
Example	<code>grep "ERROR" app.log</code>	<code>find . -name "*.log"</code>
Recursive option	<code>grep -r</code>	Recursive by default
Common use	Logs/code search	Locate files

curl vs wget

Feature	curl	wget
Main strength	API and protocol testing	File downloading
Custom HTTP methods	Strong support	Limited compared to <code>curl</code>
Headers/body	Easy with <code>-H</code> , <code>-d</code>	Less ergonomic
Recursive download	Not typical	Supported

ssh vs scp

Feature	ssh	scp
Purpose	Remote command/login	Secure file copy
Uses encryption	Yes	Yes
Based on SSH protocol	Yes	Yes

Feature	ssh	scp
Example	ssh user@host	scp file user@host:/tmp

SIGTERM vs SIGKILL

Feature	SIGTERM	SIGKILL
Number	15	9
Graceful?	Yes	No
Can be caught?	Yes	No
Cleanup possible?	Yes	No
Recommended first?	Yes	No

fork vs exec

Feature	fork	exec
Purpose	Create child process	Replace current process image
New PID?	Child gets new PID	Same PID remains
Common shell use	Create child	Run requested command
Returns?	Returns in parent and child	Returns only on failure

Archive vs Compression

Concept	Meaning	Example
Archive	Combines files	.tar
Compression	Reduces size	.gz
Archive + compression	Combines and compresses	.tar.gz

File vs Directory Permissions

Permission	On File	On Directory
Read r	Read file contents	List directory names
Write w	Modify file contents	Create/delete/rename entries
Execute x	Run as program/script	Enter/traverse directory

9. Common Mistakes

- Thinking the shell and kernel are the same.
- Using **cat** on very large log files.
- Forgetting that **cd** changes shell state and is usually built-in.

- Using `chmod 777` to "fix" permission problems without understanding security risk.
- Confusing `grep` and `find`.
- Forgetting quotes around patterns with spaces.
- Running `kill -9` immediately instead of trying graceful termination.
- Thinking `tar` always means compression.
- Not understanding `stdin`, `stdout`, and `stderr`.
- Assuming a command in a pipeline runs only after the previous command fully completes.
- Confusing a process ID with a port number.
- Thinking public SSH keys must be kept secret. The private key must be secret.
- Not knowing that directory execute permission is required to enter a directory.
- Forgetting that environment variables are inherited by child processes, not automatically by parent processes.
- Treating shell wildcards and regex as the same thing. `*.log` in shell globbing is not the same as regex `.*\.log`.

10. Edge Cases / Special Cases

Hidden Files

Files starting with `.` are hidden from normal `ls`.

```
ls -a
```

File Names With Spaces

Use quotes:

```
cat "my file.txt"
```

Or escape spaces:

```
cat my\ file.txt
```

`grep` Pattern vs Shell Expansion

This can fail unexpectedly:

```
grep *.log file.txt
```

The shell may expand `*.log` before `grep` sees it. Use quotes when you mean a pattern:

```
grep "*.log" file.txt
```

Directory Execute Permission

A directory may have read permission but still not allow entering if execute permission is missing.

```
r-- on directory: can list names if accessible  
x on directory: can traverse into it
```

kill Without Signal

This:

```
kill 1234
```

defaults to `SIGTERM`, not `SIGKILL`.

curl Output

By default, `curl` writes response body to stdout.

Save to file:

```
curl -o page.html https://example.com
```

wget Saves Files by Default

Unlike `curl`, `wget` `URL` usually saves the downloaded file directly.

tar Option Order

Common forms:

```
tar -czvf app.tar.gz app/  
tar -xzvf app.tar.gz
```

The `-f` option expects the archive file name next.

Zombie Processes

You cannot fix a zombie by killing the zombie itself because it has already exited. The parent must reap it, or the parent must exit so another process can adopt and reap it.

ps aux | grep name Shows grep Itself

This command may show the grep process too.

Common workaround:

```
ps aux | grep '[n]ginx'
```

find Can Be Expensive

Running find / may scan the whole file system and hit permission errors. Narrow the path when possible.

11. How to Explain in Interview

Linux shell is a command-line interface used to interact with the operating system. Commands like ls, grep, find, ps, curl, and ssh help navigate files, search logs, manage processes, test APIs, and work on remote servers. Internally, many commands work by making system calls such as open, read, write, fork, exec, wait, and kill. The shell parses the command, sets up input/output, starts processes, and the kernel performs privileged operations safely.

12. Quick Revision Notes

Key Definitions

Term	Quick Meaning
Shell	Command interpreter
Terminal	UI for command input/output
Kernel	Core OS resource manager
Process	Running program instance
PID	Process ID
System call	User program request to kernel
File descriptor	Integer handle to open file/input/output
Signal	Notification sent to process

Important Commands

Command	Purpose
pwd	Print current directory
cd	Change directory
ls	List files

Command	Purpose
<code>cat</code>	Print/concatenate files
<code>less</code>	View large files
<code>grep</code>	Search text
<code>find</code>	Search files
<code>chmod</code>	Change permissions
<code>ps</code>	Show processes
<code>kill</code>	Send signal
<code>curl</code>	Make URL/API requests
<code>wget</code>	Download files
<code>ssh</code>	Remote login
<code>scp</code>	Secure file copy
<code>tar</code>	Archive/extract files

Must-Remember Facts

- `cd` is a shell built-in because it changes shell state.
- `grep` searches contents; `find` searches file paths/metadata.
- `kill` sends signals; default is `SIGTERM`.
- `kill -9` is forceful and should not be the first option.
- `chmod 755` means `rxwxr-xr-x`.
- `chmod 644` means `rw-r--r--`.
- `stdin = 0, stdout = 1, stderr = 2`.
- `>` overwrites, `>>` appends.
- `|` connects stdout of one command to stdin of another.
- `fork` creates a child process; `exec` runs a new program in the current process.
- `tar` archives; `gzip` compresses.

Interview Traps

- Shell is not the kernel.
- Terminal is not the same as shell.
- Public key can be shared; private key must be secret.
- Directory execute permission is needed to enter a directory.
- `exec` does not create a new process by itself.
- `SIGKILL` cannot be caught or handled.

13. Practice Tasks

Beginner Tasks

1. Print your current directory using `pwd`.

2. Create a folder, enter it using `cd`, and verify with `pwd`.
3. List files using `ls`, `ls -l`, and `ls -a`.
4. Display a small file using `cat`.
5. Open a large file using `less` and search inside it using `/`.

Search and Logs

1. Find all `.log` files under the current directory:

```
find . -type f -name "*.log"
```

2. Search for errors in a log:

```
grep -i "error" app.log
```

3. Search recursively for TODO comments:

```
grep -rn "TODO" .
```

4. Find files modified in the last day:

```
find . -type f -mtime -1
```

Permissions

1. Create a script file.
2. Try running it before execute permission.
3. Add execute permission:

```
chmod +x script.sh
```

4. Compare:

```
chmod 644 file.txt  
chmod 600 secret.txt  
chmod 755 script.sh
```

Process Management

1. Start a long-running process:

```
sleep 1000
```

2. In another terminal, find it:

```
ps aux | grep sleep
```

3. Terminate it gracefully:

```
kill -15 <PID>
```

Networking

1. Check response headers:

```
curl -I https://example.com
```

2. Download a file using `wget`.
3. Send a POST request to a local API using `curl`.
4. SSH into a remote or local VM if available.
5. Copy a file using `scp`.

Archives

1. Create an archive:

```
tar -czvf project.tar.gz project/
```

2. List archive contents:

```
tar -tzvf project.tar.gz
```

3. Extract it:

```
tar -xzvf project.tar.gz
```

System Call Thinking Exercise

Explain which system calls are likely used by each command:

Command	Likely System Calls
cat file.txt	open, read, write, close
ls -l	open, getdents/readdir, stat, write
cd /tmp	chdir
chmod 755 a.sh	chmod
ps aux	Reads process info, commonly from /proc on Linux
kill 1234	kill
curl URL	socket, connect, read, write

14. Final Cheat Sheet

Core Definition

Linux shell commands are text commands used to control files, processes, networking, permissions, and remote machines. System calls are the low-level kernel interfaces that make these operations possible.

Why It Matters

SDEs use Linux shell skills to debug servers, inspect logs, manage processes, test APIs, transfer files, automate tasks, and understand OS behavior.

Most Asked Questions

Question	One-Line Answer
What is a shell?	A command interpreter that runs commands and scripts.
What happens when you run <code>ls</code> ?	Shell forks, child execs <code>ls</code> , <code>ls</code> uses system calls, parent waits.
Why is <code>cd</code> built-in?	It must change the current shell process's directory.
<code>grep</code> vs <code>find</code> ?	<code>grep</code> searches contents; <code>find</code> searches files/metadata.
<code>kill -15</code> vs <code>kill -9</code> ?	<code>SIGTERM</code> is graceful; <code>SIGKILL</code> is forceful.
<code>curl</code> vs <code>wget</code> ?	<code>curl</code> is great for API requests; <code>wget</code> is great for downloads.
<code>tar</code> vs <code>gzip</code> ?	<code>tar</code> archives; <code>gzip</code> compresses.
<code>fork</code> vs <code>exec</code> ?	<code>fork</code> creates a process; <code>exec</code> replaces process image.

Common Comparisons

Comparison	Key Difference
Shell vs Terminal	Shell interprets; terminal displays.

Comparison	Key Difference
Shell vs Kernel	Shell requests; kernel manages resources.
Absolute vs Relative Path	Root-based vs current-directory-based.
File vs Directory Permission	Directory x means traverse.
stdout vs stderr	Normal output vs error output.

One-Line Interview Answer

The Linux shell lets me control the OS through commands for files, processes, networking, permissions, and remote access, while system calls like **open**, **read**, **write**, **fork**, **exec**, and **kill** are the kernel-level mechanisms that actually perform those operations.